Containerization

Namespaces, cgroups, and chroot

By: Clay Molitor

What is Containerization?

- It’s like virtualization, but it’s not.
- Containers share most of their OS with their host.
- Virtualization runs a new OS for each system.

Control Groups

Control Groups

- **cgroups v1:** older, largely deprecated.

```
├── memory
│   ├── a
│   └── b
└── cpu
    ├── a
    └── b
```

- **cgroups v2:**

```
├── a
│   ├── memory
│   └── cpu
└── b
    ├── memory
    └── cpu
```

Control Groups

- Each process is in exactly one cgroup (v2).
- Can be used with systemd:

```
> sudo systemd-run --scope -p MemoryMax=1G -p CPUQuota=100% bash
```

- cgroups control: **cpu**, **memory**, io , pids count, cpuset, hugepage allocation, rdma, and misc.

Namespaces

Namespaces

- **UTS:** Isolates hostname and domain name.
- **user:** Maps namespaced users to parent’s users.
- **mount**: Isolates mounts between parent and child.
- **PID:** Start a new process tree.
- **network:** Network interfaces, IP routing tables, firewall rules, sockets, etc
- **cgroups:** Inherent parent's cgroups, new cgroups are nested in host cgroup.
- **time:** Creates distinct CLOCK_BOOTTIME, not a unique system time.
- **IPC:** Isolates POSIX and System V message queues.

Namespaces - UTS

Provides hostname isolation.

Allows unprivileged users to set hostname (in child namespace).

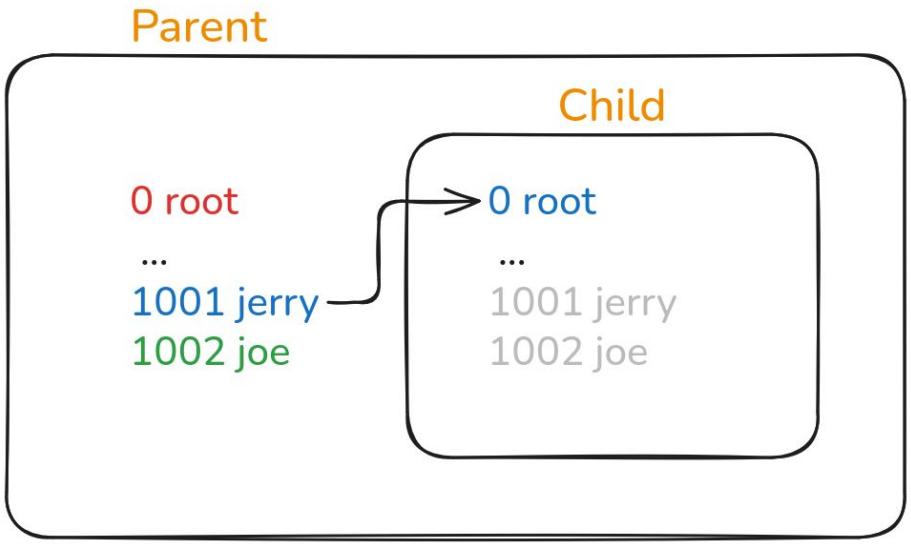
```
> hostname
0e521c9ece1c

> hostname jerry-pc
hostname: you must be root to change the host name

> unshare --map-root-user --uts bash -c ‘hostname joe-pc && hostname’ && hostname
joe-pc
0e521c9ece1c
```

Namespaces - User

Does not provide additional permissions.
`--map-root-user` can bypass `user == "root"` checks.



Namespaces - User


```
> unshare --user --map-root-user id
uid=0(root) gid=0(root) groups=0(root)
```

```
> unshare --user --map-root-user touch /file
touch: cannot touch '/file': Permission denied
```

* `--user` is assumed by `-map-root-user`

Namespaces - Mount


```
> mount -l
overlay on / type overlay (rw,...)
proc on /proc type proc (rw,...)
tmpfs on /dev type tmpfs (rw,...)
devpts on /dev/pts type devpts (rw,...)
sysfs on /sys type sysfs (rw,...)
cgroup on /sys/fs/cgroup type cgroup2 (rw,...)
mqueue on /dev/mqueue type mqueue (rw,...)
shm on /dev/shm type tmpfs (rw,...)
devpts on /dev/console type devpts (rw,...)

> unshare --mount mount -l
[same as above]
```

Namespaces - Mount

Child inherits parent's mounts.

In docker/podman --privileged /sys mounts are rw, ro otherwise.

Clear mounts by chrooting, unmounting parent mounts, or mounting over them.

```
> mkdir ~/mymount
> unshare --map-root-user --mount bash -c ' \
    mount --bind ~/lab ~/mymount && \
    mount -l | grep "mymount" && \
    ls ~/mymount'
overlay on /home/jerry/mymount type overlay (rw, ...)
A  B  C  Ch  D  flag_private  flag_public
```

Namespaces - Mount

Child inherits parent's mounts.

In docker/podman --privileged /sys mounts are rw, ro otherwise.

Clear mounts by chrooting, unmounting parent mounts, or mounting over them.

```
> mkdir ~/mymount
> unshare --map-root-user --mount bash -c ' \
    mount --bind ~/lab ~/mymount && \
    mount -l | grep "mymount" && \
    ls ~/mymount'
overlay on /home/jerry/mymount type overlay (rw, ...)
A  B  C  Ch  D  flag_private  flag_public
```

Namespaces - PID

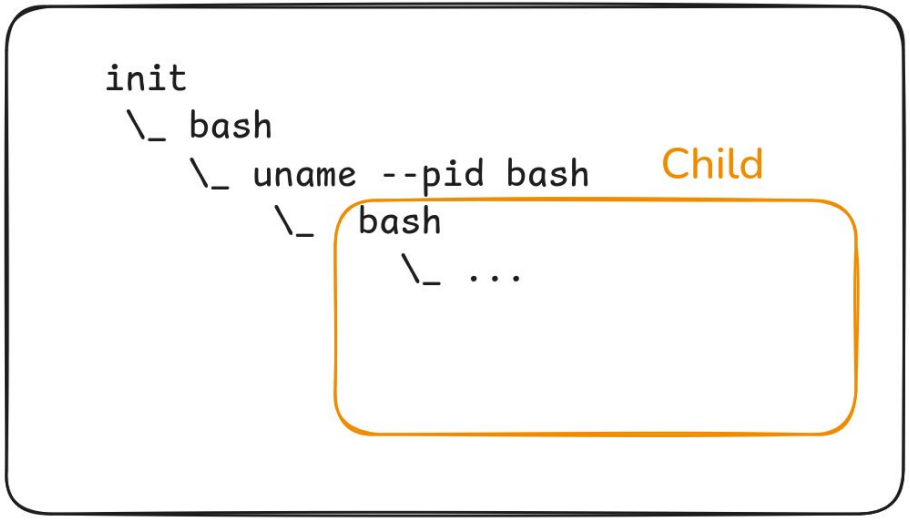
Child must remount /proc. Limited usefulness without --mount and --fork

```
> unshare --map-root-user --pid --fork bash -c 'ps a'
    PID TTY      STAT   TIME COMMAND
      1 pts/0    Ss+    0:00 /bin/bash /entrypoint.sh /bin/bash
     39 pts/0    S+     0:28 /bin/bash /helper-B.sh
     40 pts/0    S+     0:00 /bin/bash /helper-C.sh
     ...
 142687 pts/4    S+     0:00 unshare --map-root-user --mount --pid --fork sh -c ps a
 142688 pts/4    R+     0:00 ps a


> unshare --map-root-user --mount --pid --fork bash -c 'mount -t proc proc /proc &&
ps a'
PID TTY      STAT   TIME COMMAND
  1 pts/4    R+     0:00 ps a
```

Namespaces - PID

Parent can see child's proc tree.



Namespaces - PID

Parent:

```
> ps a --forest
    109 pts/4    Ss     0:00 -bash
 127579 pts/4    S      0:00  \_ unshare --map-root-user --mount --pid --fork sh
 127580 pts/4    S+     0:00      \_ sh
 130178 pts/4    S      0:00          \_ sleep 1337
```

Child:

```
> ps a --forest
      1 pts/4    S      0:00 sh
     13 pts/4    S      0:00  \_ sleep 1337
```

Namespaces - Network

Network namespaces isolate:

- Network devices
- Listening and bound ports and sockets
- IPv4/6 stacks
- Routing tables
- Firewall rules
- /proc/net (/proc/pid/net)
- /sys/class/net
- /proc/sys/net
- And more!

Namespaces - Network

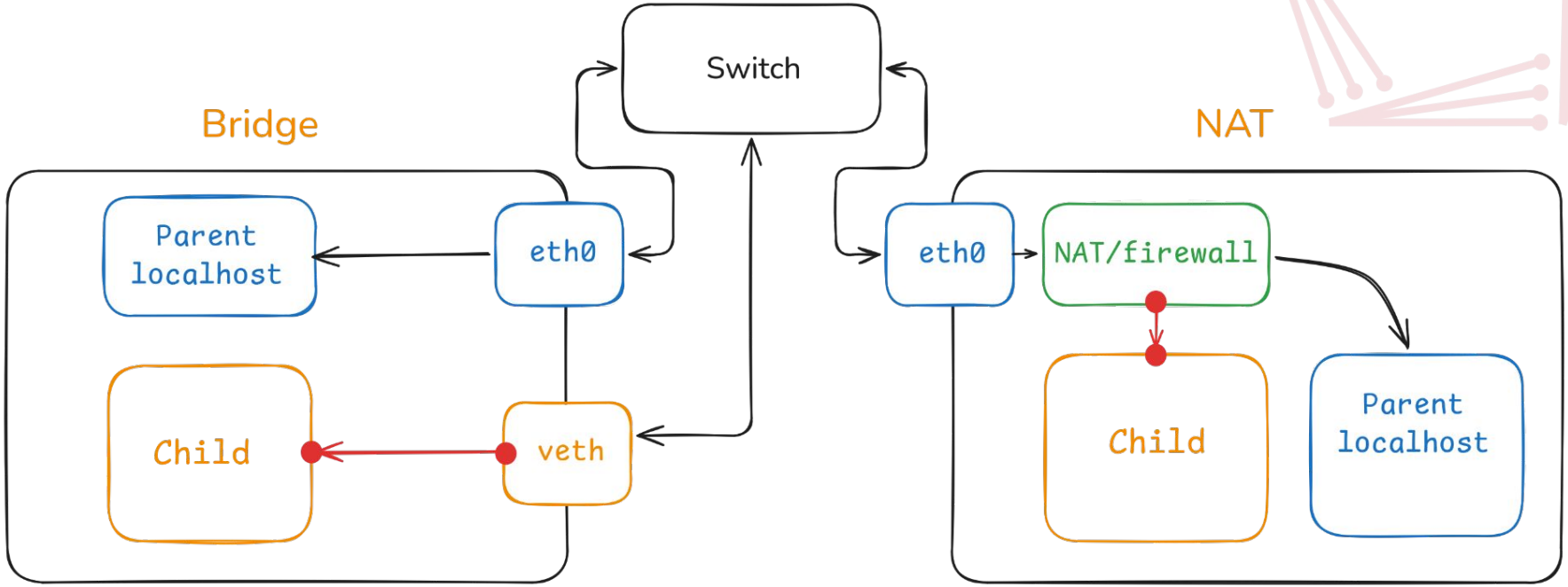
- Network devices (interfaces) can only live in one network namespace at a time.
- If you move eth0 into a new namespace, the parent namespace loses internet.
- Virtual interfaces are created in pairs and send traffic to the other pair.
- One side of the virtual interface can be namespaced and the other attach to eth0.
- Virtual interfaces can be connected to existing interfaces through NAT or Bridging.

Namespaces - Network



- **Bridging:** L2, same broadcast address, new IP for new interfaces.
- **NAT:** L3, new subnet for new interfaces.

Namespaces - Network



Namespaces - Others

- **cgroups:**
  - Child inherits the parents cgroups
- **time:**
  - Only isolates boot/up time. Time zones live in /etc, system time on the rtc.
- **IPC:** Isolates POSIX and System V message queues.
  - Isolates message queues. Neither parent nor child can see the other’s queue.

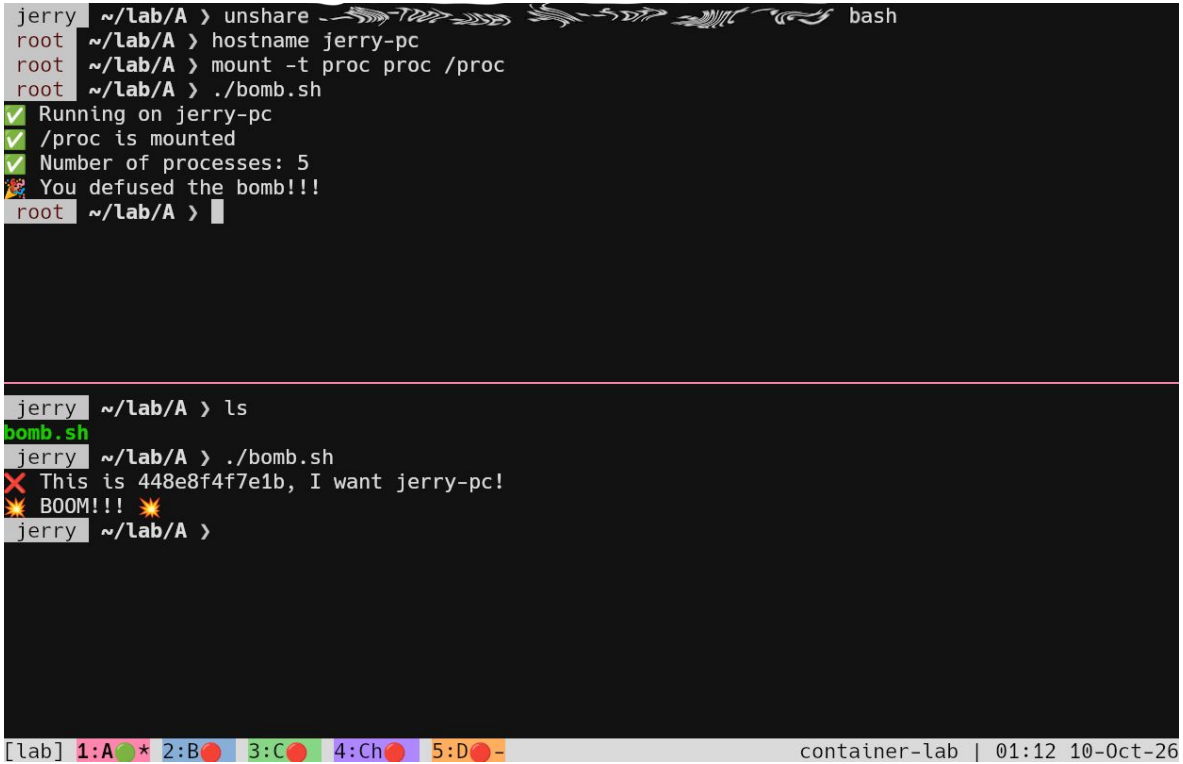
Container Lab Walkthrough

```
git clone https://github.com/molitorclay/container-lab

cd container-lab

./build.sh
```

Lab - A (PID)



tmux:

hotkeys:
Next tab: CRTL+a, n
Next pane: CRTL+a, arrow

Or left click on panes and tabs to navigate

Mount



- Chroot and pivot-root loses access to /bin, /usr, /etc, /proc, /sys, /dev and all other root dirs.
- Either mount the parent’s /bin /etc or create new ones.
- Chroot and pivot-root are similar, but chroot can be escaped by a privileged process.
- Chroot is used in the lab because it is simpler, but pivot-root should be used if security is a concern.

Chroot

```
 jerry  ~/lab/B ❯ ./install-busybox.sh
 jerry  ~/lab/B ❯ ls
bin  bomb.sh  etc  install-busybox.sh
 jerry  ~/lab/B ❯ unshare --map-root-user chroot . /bin/sh -l
/ # ls
bin                 bomb.sh             etc                 install-busybox.sh
/ # pwd
/
/ # sh --version
/ #
```

```
 jerry  ~/lab/B ❯ pwd
/home/jerry/lab/B
 jerry  ~/lab/B ❯ sh --version
GNU bash, version 5.3.9(1)-release (x86_64-redhat-linux-gnu)
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>

This is free software; you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
 jerry  ~/lab/B ❯
```

```
[lab] 1:A🟢- 2:B🔴* 3:C🔴 4:Ch🔴 5:D🔴                    container-lab | 02:30 10-Oct-26
```

tmux:

hotkeys:
Next tab: CRTL+a, n
Next pane: CRTL+a, arrow

Or left click on panes and tabs to navigate

Pivot-root

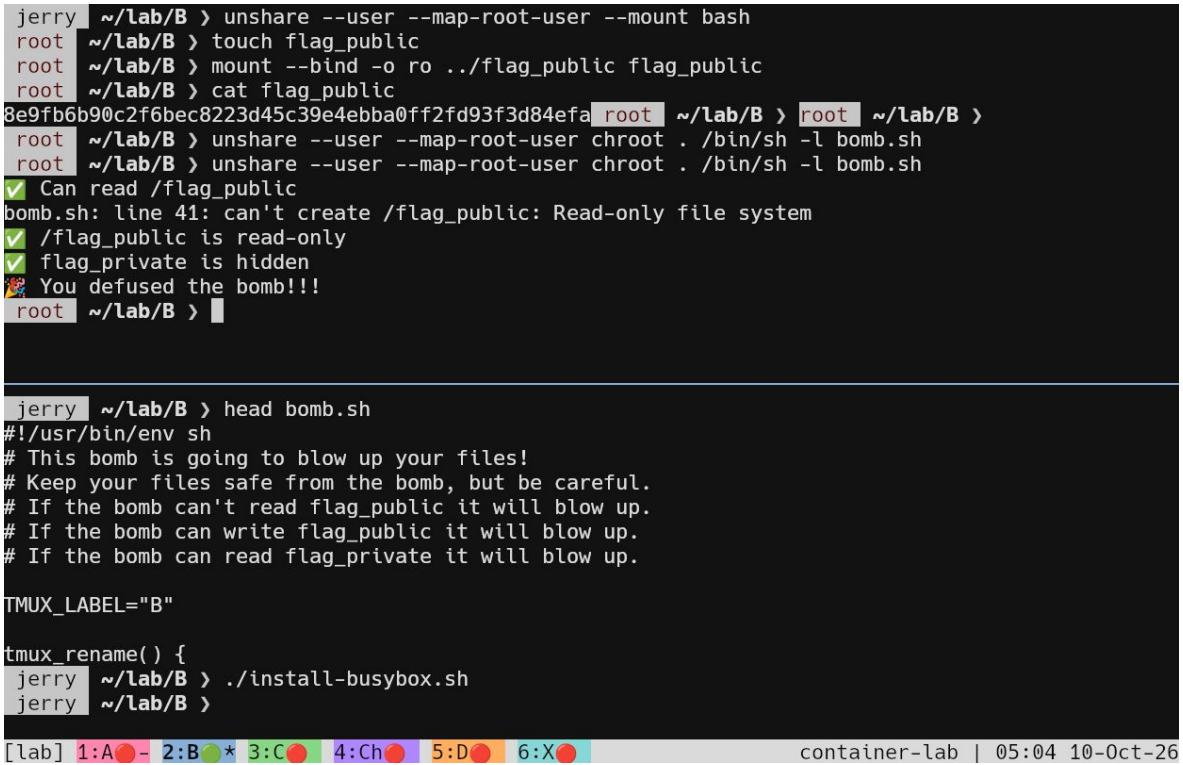
```
jerry ~/lab/B ❯ unshare --map-root-user --mount sh -c '
    mount --make-rprivate /
    mount --bind /home/jerry/lab/B /home/jerry/lab/B
    mkdir -p /home/jerry/lab/B/old_root
    pivot_root /home/jerry/lab/B /home/jerry/lab/B/old_root
    /bin/sh -l
'
/old_root/home/jerry/lab/B # cd /
/ # ls
bin                etc                old_root
bomb.sh            install-busybox.sh
/ # ls old_root/
Dockerfile.base64  entrypoint.sh      lib64              run                usr
afs                etc                media              sbin               var
bin                helper-B.sh        mnt                srv
boot               helper-C.sh        opt                sys
depth              home               proc               tmp
dev                lib                root               tmux-launch.sh
/ #
[lab] 1:A🟢- 2:B💥*Z 3:C🔴 4:Ch🔴 5:D🔴                    container-lab | 03:53 10-Oct-26
```

tmux:

hotkeys:
Next tab: CRTL+a, n
Next pane: CRTL+a, arrow

Or left click on panes and tabs to navigate

Lab - B (Mount)

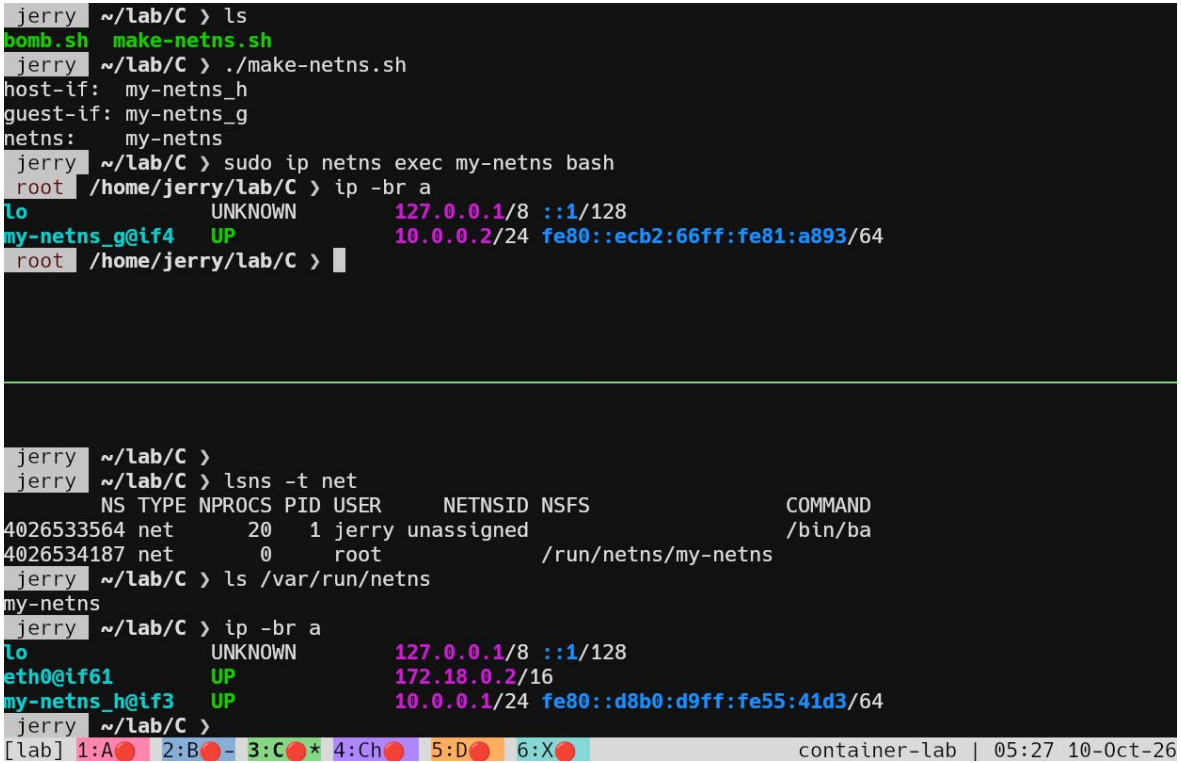


tmux:

hotkeys:
Next tab: CRTL+a, n
Next pane: CRTL+a, arrow

Or left click on panes and tabs to navigate

Lab - C (Net)



tmux:

hotkeys:
Next tab: CRTL+a, n
Next pane: CRTL+a, arrow

Or left click on panes and tabs to navigate

Overlays


```
> tree
├── 1
│   └── a_file
├── 2
│   └── bee_file
├── 3
│   └── a_file
└── 4
    └── sea_file

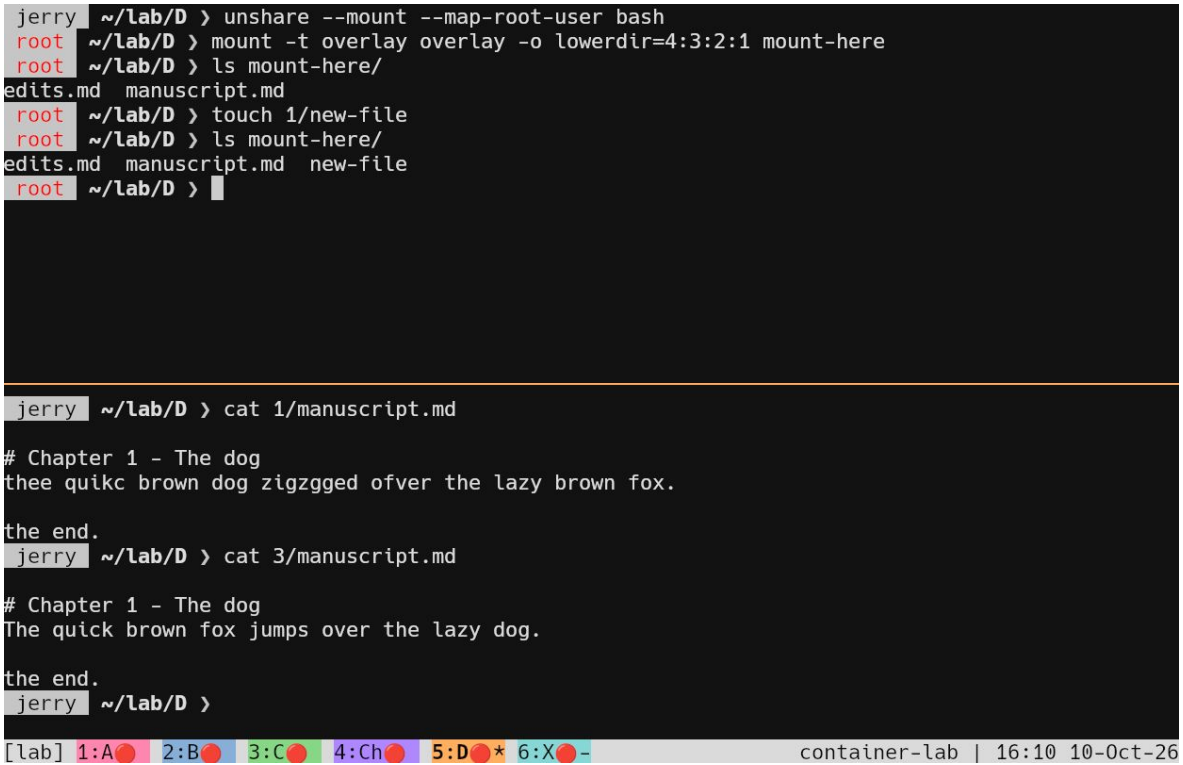
> mount -t overlay overlay -o lowerdir=4:3:2:1 my-overlay

> ls my-overlay
a_file  bee_file  sea_file
```

Overlay

- Each Dockerfile COPY, RUN, etc creates a new layer.
- These layers can be explored with tools like dive.
- Secrets added to these layers stay in middle layers, even if they are not visible in the final overlay.

Lab - D (Overlay)



tmux:

hotkeys:
Next tab: CRTL+a, n
Next pane: CRTL+a, arrow

Or left click on panes and tabs to navigate

Tools

Tools - engines

- **Docker:** most popular; daemon (dockerd) talks to containerd, which talks to runc.
- **Podman:** Docker-compatible CLI, no daemon, rootless by default; talks directly to crun.
- **nerdctl:** Docker-compatible CLI for containerd directly (no dockerd).
- **LXC / Incus:** "system containers" running a whole OS instead of one process.

Tools - runtimes

- **runc** — the reference implementation, written in Go. Default in Docker.
- **crun** — same job in C. ~2× faster startup, smaller binary, better cgroups v2. Default in Podman.
- **youki** — same again in Rust. Drop-in replacement, growing use.
- **gVisor (runsc)** — same interface, but intercepts syscalls in a userspace "guest kernel" written in Go. Container syscalls never reach the host kernel.
- **Kata Containers** — same interface, but runs the container inside a lightweight VM (QEMU or Firecracker). Full kernel isolation.

Tools - more



- **Sandboxes** (not full containers, but same primitives):

- Bubblewrap (bwrap) — unprivileged namespace sandbox; what Flatpak uses.
- systemd-nspawn — chroot-on-steroids; can boot a full OS.
- Firejail — SUID binary for desktop app isolation.

- **Image builders** (produce OCI images without needing a daemon):

- Buildah — Podman's sibling; builds from Dockerfile or shell scripts.
- BuildKit — the engine behind docker buildx; concurrent stages, secrets.
- Kaniko — builds inside a container, no daemon, no root. Designed for CI.

Thank You!